PROJECT TITLE

Job Title Prediction & Semantic Matching using Machine Learning & NLP

ORGANIZATION NAME & ADDRESS

Null Class Edtech Pvt. Ltd ,

Barur post krishnagiri , Tamil Nadu 653201

SUBMITTED BY

Mr. SHREYAS TARFE

M.Sc. (Applied Statistics)

PRN: 23060641094

SYMBIOSIS STATISTICAL INSTITUTE

॥वसुधैव कुटुम्बकम्॥

ACADEMIC YEAR 2024 - 25

Under the guidance of

Mrs. Elavarasi

Designation: COO

Email Id: training@nullclass.com

Mobile: +91 94893 38060

Contents

1. **Executive Summary** ……
2. **Study Background** ……..
3. **Aims & Objectives** ……..
4. **Methodology** ……
5. **Results** ……..
6. **Discussion and Conclusion** …….
7. **Limitations And Further Work**
8. **Acknowledgement** ……...
9. **References** ……...……
10. **Annexure – (Questionnaire & other details)** ……

Executive Summary :

This project involves an in-depth analysis of job description data with a primary focus on salary information. The dataset includes salary figures expressed as ranges, which were preprocessed to isolate minimum and maximum values. These were then converted into numerical formats and used to compute an average salary for each job posting, enabling standardized comparisons across roles.

The analysis began with thorough data preparation, including identifying missing values and ensuring consistency in salary formats. By splitting and transforming the "Salary Range" column, meaningful salary insights were extracted that could be used in further analysis.

The key objective of this project is to gain insights into compensation patterns across various job listings. These findings can support job seekers, HR departments, and market researchers by providing clear benchmarks for salaries. Python libraries such as Pandas and Scikit-learn were used for data processing and scaling, laying the groundwork for future exploration, including potential predictive salary models.

This project demonstrates the ability to clean and transform semi-structured data, perform foundational analysis, and derive insights with practical relevance in the job market landscape.

Introduction :

In the evolving landscape of employment, analyzing job market data has become increasingly important for understanding salary expectations, skill requirements, and hiring trends. However, job descriptions often come in unstructured textual formats, especially when it comes to salary ranges and role details. This project tackles that challenge by applying both data preprocessing and Natural Language Processing (NLP) techniques to extract meaningful insights from job listings.

The dataset comprises job descriptions with salary ranges presented in text formats .The initial steps focus on cleaning and converting these salary values into structured numerical columns—minimum, maximum, and average salary—for better analysis and comparison.

Beyond salary extraction, the project leverages **NLP techniques** to analyze the **textual content of job descriptions**. This includes processing job roles, qualifications, and required skills to identify common patterns and frequently mentioned keywords. Techniques such as **tokenization, stopword removal, and TF-IDF vectorization** are employed to transform unstructured text into meaningful features.

By combining traditional data analysis with NLP, this project aims to provide a comprehensive overview of the job market. The findings can support job seekers in identifying high-demand roles, help employers benchmark compensation, and contribute to a deeper understanding of industry-specific language trends.

Aim and Objective :

The primary objective of this project is to analyze job description data with a specific focus on extracting and understanding salary-related information. The project aims to convert unstructured salary ranges into structured numerical formats by identifying minimum, maximum, and average salary values.

In addition to numerical analysis, the project also utilizes Natural Language Processing (NLP) techniques to process and analyze the textual content of job descriptions. This involves extracting key skills, qualifications, and frequently mentioned terms to identify patterns in job market demands. By combining structured salary data with insights from unstructured text, the project seeks to uncover trends and relationships that can guide job seekers, employers, and analysts in making informed decisions. Furthermore, this analysis serves as a foundation for potential future applications such as predictive modeling and automated job matching systems.

About the Data :

The dataset comprises various features that represent different aspects of job postings, from job role and salary to company details and location. Below is a detailed explanation of each variable:

1. **Job Id**: A unique identifier assigned to each job listing. This helps in distinguishing and referencing individual records without ambiguity.

2. Experience: Specifies the required level of professional experience for the job. It was mentioned in ranges (e.g., "0–2 years") or as a minimum threshold (e.g., "3+ years"), and is critical for filtering jobs by candidate eligibility.

3. Qualifications: Lists the educational background or degrees required for the role, such as "Bachelor’s in Computer Science", "MBA", or specific certifications like "PMP" or "AWS Certified".

4. Salary Range: A textual representation of the expected salary for the job, often provided as a range (e.g., "$50,000 – $70,000"). This column is preprocessed to extract minimum, maximum, and average salary values for quantitative analysis.

5. Location: The city, region, or locality where the job is based. It provides insight into geographic job trends and cost-of-living considerations.

6. Country: Indicates the nation in which the job opportunity exists. It is useful for international comparison and understanding job market trends across different countries.

7. Latitude & Longitude: These numerical values represent the geographical coordinates of the job location. They enable spatial analysis and visualization on maps to observe job distribution.

8. Work Type: Describes the nature of employment, such as full-time, part-time, contractual, remote, or hybrid. It helps job seekers identify suitable working arrangements.

9. Company Size: Categorizes the company based on its workforce or scale—commonly segmented as small, medium, or large enterprises. This can be useful in analyzing trends by company size or hiring capacity.

10. Job Posting Date: The date the job was listed or made public. It is useful for time-series analysis to observe posting trends over time or filter recent opportunities.

11. Preference: Represents any specific preferences mentioned by employers, such as preferred gender, language skills, or availability, which might influence applicant selection.

12. Contact Person & Contact: These fields provide information about the person handling recruitment, typically someone from HR or a hiring manager, along with their contact details for communication.

13. Job Title: The specific designation of the job (e.g., “Data Scientist”, “Marketing Manager”). This is a crucial feature for categorizing and analyzing roles.

14. Role: A broader classification of the job function such as IT, HR, Finance, etc., which helps in grouping similar job titles under a unified domain.

15. Job Portal: Identifies the platform or website where the job was originally posted, such as LinkedIn, Indeed, Naukri, etc., offering insights into platform-based posting trends.

16. Job Description: A textual summary of the job's responsibilities, required qualifications, and other relevant details. This unstructured field is key for applying NLP techniques to extract insights.

17. Benefits: Mentions perks and advantages provided by the employer (e.g., health insurance, remote work, bonuses), which can influence job preference.

18. Skills: Lists technical or soft skills required for the role (e.g., Python, communication, leadership). This is important for keyword extraction and matching jobs to candidate profiles.

19. Responsibilities: Describes the day-to-day duties expected from the employee. It provides deeper insight into role expectations and workload.

20. Company: The name of the organization posting the job. It helps in brand-based or reputation-based filtering of opportunities.

21. Company Profile: A short description of the company, including industry, mission, and culture. It can help job seekers understand the company's background before applying.

Methodology :

Data Collection

The dataset containing job descriptions and salary information was imported from a CSV file named `job_descriptions.csv`. This dataset served as the primary source for all further analysis.

Initial Data Exploration

Basic exploratory analysis was performed to understand the structure and contents of the dataset. This included viewing column names, checking data types, and identifying missing values using methods like `.info()`, `.describe()`, and `.isnull().sum()`.

Data Cleaning and Preprocessing

The "Salary Range" column was cleaned and split into two separate columns representing the lower and upper bounds of the salary. These values were converted from textual formats into numeric values and scaled to represent actual salary figures. A new column for the average salary was also computed.

Dropping Unnecessary Columns

Columns that were not relevant to the core analysis or contained too many missing or unstructured values were removed to streamline the dataset. This helped reduce noise and improve processing efficiency.

Encoding Categorical Variables

Categorical features, such as job titles or industries (if present), were encoded into numerical formats using **Label Encoding**. This step ensured compatibility with analysis tools and future machine learning models.

Feature Engineering

New features such as `Salary_Min`, `Salary_Max`, and `Salary_Avg` were created for detailed analysis. Additional columns may have been generated during text processing.

Natural Language Processing (NLP)
The textual job descriptions were processed using NLP techniques. This included:

- **Tokenization** to break text into individual words
- **Stopword removal** to eliminate common, non-informative words
- **Vectorization** using techniques such as TF-IDF to convert text into numerical features
- **Keyword extraction** to identify frequently occurring terms or required skills

Exploratory Data Analysis (EDA)
Visualizations and summary statistics were used to analyze salary distributions, identify patterns in job postings, and explore relationships between job roles, skills, and salaries.

Scaling and Standardization
Numerical features were standardized using tools like `StandardScaler` to prepare the data for potential future modeling.

Insights Generation
Key trends were identified based on salary ranges and commonly required skills. These insights aim to support decision-making for job seekers and recruiters.

Documentation and Reporting
The results were organized, and key steps were documented for clarity. The entire workflow was structured to be reproducible and interpretable.

Results :

Descriptive Statistics and Data Cleaning

- After preprocessing, including removing duplicates and handling missing values, the dataset was refined to ensure quality analysis.
- Salary values originally in textual ranges (e.g., "₹10,000 - ₹20,000") were successfully transformed into numerical format (Salary_Avg) for meaningful analysis.
- Unnecessary columns such as Contact, Contact_Person, and Job_Portal were removed, and categorical variables like Country, Role, and Company_Size were encoded for modeling purposes.

---

2. Salary Distribution and Trends

- The salary distribution showed a right-skewed pattern, indicating that while most jobs offer moderate pay, a few roles command significantly higher salaries.
- On average, jobs with higher experience requirements and advanced qualifications tend to offer better compensation.

- Visual analysis using boxplots and histograms confirmed that managerial roles and technical positions such as Data Scientists and Software Engineers are among the top-paying roles.

---

3. Correlation Analysis

- The heatmap visualization revealed that Salary_Avg has a strong positive correlation with Salary_Max (0.98) and a weak positive correlation with Salary_Min (0.21).

- Variables such as Job_Id, Company_Size, latitude, and longitude showed negligible correlation with Salary_Avg, suggesting they are not strong indicators of compensation in this dataset.

- This suggests that while the max salary value helps define the average, external location-based factors play a limited role.

---

4. NLP Analysis on Job Descriptions

```
# Example usage
user_input = "Looking for a mid-level Python developer with experience in data analysis"
matched_job = match_job_description(user_input)
print(f"Best matched job: {matched_job}")
```

```
Best matched job: Data Scientist
```

```
# Example usage
user_input = "Looking for a mid-leve
matched_job = match_job_description(user_input)
print(f"Best matched job: {matched_job}")
```

```
(variable) user_input: Literal['Looking for a mid-level Python developer with expe…']
```

```
Best matched job: Mechanical Designer
```

```
# Example usage
user_input = "Looking for a mid-level data analyst with experience in machine learning"
matched_job = match_job_description(user_input)
print(f"Best matched job: {matched_job}")
```

```
Best matched job: Data Scientist
```

```
# Example usage
user_input = "Looking for a digital marketing expert with skills in SEO, social media campaigns, content marketing"

matched_job = match_job_description(user_input)
print(f"Best matched job: {matched_job}")
```

Activate Windows
Go to Settings to activat

```
Best matched job: Marketing Manager
```

- Natural Language Processing techniques, including tokenization, stopword removal, and TF-IDF vectorization, were used on the Job_Description column.
- Word cloud and frequency distribution revealed that skills like **Python**, **communication**, **Excel**, **data analysis**, and **teamwork** appeared most frequently.
- These patterns indicate a strong emphasis on technical skills and soft skills in most job roles.

---

5. Role and Skills Mapping

- Certain roles such as **Business Analyst**, **HR Executive**, and **Software Developer** had clearly defined sets of frequently associated skills.
- Classification of roles using description-based keywords helped in grouping job types and understanding the required competencies.

Conclusion :

This project provided a comprehensive analysis of job descriptions with a focus on understanding salary dynamics and the skills landscape in the current job market. Through careful data preprocessing and transformation, including the extraction and conversion of salary ranges into numerical features, we were able to create a structured dataset ready for in-depth exploration.

Furthermore, the heatmap of correlations revealed key numerical features that influence salary levels, providing actionable insights for job seekers and HR professionals. Features such as experience, qualifications, and company size showed varying degrees of association with average salary, highlighting the multifaceted nature of compensation decisions.

Additionally, categorical variables like job roles, company types, and work types were encoded and analyzed, offering a more complete picture of the job environment. These analyses allowed for segmentation and trend discovery across different sectors and geographies.

The application of Natural Language Processing (NLP) played a crucial role in analyzing unstructured job description text. By implementing techniques like tokenization, stopword removal, and TF-IDF vectorization, we extracted meaningful patterns and frequently demanded skills across various job roles. This helped in identifying the most valued competencies in the job market and understanding the linguistic trends in job advertisements.

Overall, this project not only strengthened understanding of job market trends but also demonstrated how structured and unstructured data can be integrated to deliver impactful business insights. The findings from this study can aid job seekers in aligning their skills with market demands, support recruiters in crafting better job postings, and assist companies in benchmarking their compensation strategies effectively.

Limitations and Further Work:

Despite the valuable insights gained from this study, there are certain limitations that should be acknowledged. Firstly, the dataset may not be exhaustive or fully representative of the entire job market, as it is dependent on the availability and completeness of job listings from specific portals. This could introduce selection bias and limit the generalizability of the findings.

In terms of text analysis, the Natural Language Processing techniques used were limited to basic preprocessing and TF-IDF vectorization. More advanced methods such as Named Entity Recognition (NER), topic modeling, or BERT embeddings could provide deeper understanding of the job descriptions, skills, and roles mentioned.

Another constraint lies in the handling of categorical features. While label encoding was used to convert these into numerical form, this approach may oversimplify relationships and does not account for potential ordinal or hierarchical structures. One-hot encoding or target encoding could be explored to preserve more information.

Future Work

To enhance the project further, future efforts can be directed toward incorporating machine learning models to predict salaries or classify job roles based on descriptions and other attributes. Including external data sources, such as economic indicators or industry benchmarks, could also improve the robustness of the analysis.

Advanced NLP techniques such as word embeddings (Word2Vec, GloVe), topic modeling (LDA), or transformer-based models (e.g., BERT) could be employed to extract deeper semantic meaning from job descriptions. Additionally, integrating a time-series component to analyze how job trends, skills, or salary ranges evolve over time would add a dynamic perspective to the study.

Acknowledgement :

I would like to express my heartfelt gratitude to my faculty mentor and guide for their continuous support, expert guidance, and thoughtful feedback throughout this project. Their mentorship was pivotal in helping me navigate complex analytical challenges and in refining the overall structure and methodology of the work. Each discussion with them provided clarity and direction, allowing me to approach the project with greater confidence and precision.

I am also thankful to Symbiosis Statistical Institute for fostering a rich academic atmosphere and providing access to the necessary tools, resources, and learning opportunities that enabled me to carry out this project effectively. The institute's emphasis on academic rigor and practical learning played a significant role in shaping my approach to problem-solving and data analysis.

Additionally, I deeply appreciate the support and collaboration of my peers, whose constructive suggestions, critical perspectives, and constant encouragement kept me motivated and enriched my learning experience throughout the project.

References :

- Computational Job Market Analysis with Natural Language Processing
Author: Mike Zhang
Published: April 2024

- Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings
Authors: Elena Senger, Mike Zhang, Rob van der Goot, Barbara Plank
Published: February 2024

- JobBERT: Understanding Job Titles through Skills
Authors: Jens-Joris Decorte, Jeroen Van Hautte, Thomas Demeester, Chris Develder
Published: September 2021

- LLM4Jobs: Unsupervised Occupation Extraction and Standardization Leveraging Large Language Models
Authors: Nan Li, Bo Kang, Tijl De Bie
Published: September 2023

Annexure :

```
df = pd.read_csv('job_descriptions.csv')
print(df.info)
print(df.describe)
df.columns
print(df.head)
# missing values
df.isnull().sum()
```

```
#Preprocessing of the data

# Split the 'Salary Range' column and extract numeric values for lower and
upper bounds
salary_bounds = df['Salary Range'].str.extract(r'(?i)\$?(\d+)[k]-
\$?(\d+)[k]')

# Renameing columns for clarity
salary_bounds.columns = ['LowerBound', 'UpperBound']

# Converting to numeric and scale to full salary amount
salary_bounds = salary_bounds.apply(pd.to_numeric) * 1000

# Assigning new columns to the original DataFrame
df['Salary_Min'] = salary_bounds['LowerBound']
df['Salary_Max'] = salary_bounds['UpperBound']

# Computing average salary
df['Salary_Avg'] = df[['Salary_Min', 'Salary_Max']].mean(axis=1)

df['Salary_Avg']
```

```
import seaborn as sns
import matplotlib.pyplot as plt
correlation = df.corr(numeric_only=True)

# Plot heatmap of the target variable with others
```

```
plt.figure(figsize=(10, 6))
sns.heatmap(correlation[['Salary_Avg']].sort_values(by='Salary_Avg',
ascending=False),
            annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation of Salary_Avg with Other Variables')
plt.show()
import pandas as pd
from sklearn.preprocessing import StandardScaler

# Extracting salary range
df[['Min Salary', 'Max Salary']] = df['Salary
Range'].str.extract(r'\$?(\d+)[kK]-\$?(\d+)[kK]')
df['Min Salary'] = pd.to_numeric(df['Min Salary']) * 1000
df['Max Salary'] = pd.to_numeric(df['Max Salary']) * 1000

# scaling
scaler = StandardScaler()
salary_cols = ['Min Salary', 'Max Salary']

# Ensuring the columns exist before transforming
if all(col in df.columns for col in salary_cols):
    df[salary_cols] = scaler.fit_transform(df[salary_cols])
else:
    print("Required columns not found in DataFrame.")

df.head()
```

```
# Defining a list of irrelevant or redundant columns
unnecessary_cols = [
    'Company Profile', 'Contact', 'Contact Person',
    'Job Id', 'Job Posting Date', 'latitude',
    'longitude', 'Job Portal', 'Salary Range'
]

# Remove those columns from the DataFrame
df = df.drop(labels=unnecessary_cols, axis=1)

df.head()
# unique job titles using set for variation
job_titles_set = set(df['Job Title'])


unique_titles_list = list(job_titles_set)
```

```
# job titles
for title in unique_titles_list:
    print(title)

from sklearn.preprocessing import LabelEncoder

label_enc = LabelEncoder()
from sklearn.preprocessing import LabelEncoder

# List of categorical columns to encode
cat_columns = [
    'Experience', 'Qualifications', 'location', 'Country',
    'Work Type', 'Company Size', 'Preference', 'Job Title',
    'Role', 'Job Description', 'Benefits', 'skills',
    'Responsibilities', 'Company'
]
# Dictionary to store label mappings
label_mappings = {}

# Create a LabelEncoder instance
encoder = LabelEncoder()

# Encode each column
for col_name in cat_columns:
    df[col_name] = encoder.fit_transform(df[col_name].astype(str))
    label_mappings[col_name] = dict(zip(range(len(encoder.classes_)),
encoder.classes_))

label_mappings['Experience']
```

```
from sklearn.model_selection import train_test_split

# Separate the target column from the predictors
features = df.drop(columns=['Job Title'])
target = df['Job Title']

# Split the data into training and testing sets (80% train, 20% test)
X_train_set, X_test_set, y_train_set, y_test_set = train_test_split(
    features, target, test_size=0.2, random_state=42
)

from sklearn.ensemble import RandomForestClassifier
```

```python
# Initialize the random forest classifier with all CPU cores
rf_classifier = RandomForestClassifier(n_jobs=-1, random_state=42)

# Train the model using the training data
rf_classifier.fit(X_train_set, y_train_set)
from sklearn.metrics import accuracy_score

# Generate predictions on the test set
y_pred = rf_classifier.predict(X_test_set)

# Calculate the accuracy of the model
model_accuracy = accuracy_score(y_test_set, y_pred)

# Display shapes and accuracy
print(f"Actual labels shape: {y_test_set.shape}")
print(f"Predicted labels shape: {y_pred.shape}")
print(f"Model Accuracy: {model_accuracy:.4f}")

print(df.corr(numeric_only=True)['Job
Title'].sort_values(ascending=False))
```

```python
# Import fuzzywuzzy
!pip install fuzzywuzzy[speedup]
from fuzzywuzzy import process

def fuzzy_encode(user_input, target_column):
    # Check if the column is in the mapping dictionary
    if target_column not in label_mappings:
        raise KeyError(f"No encoding found for column: {target_column}")

    # Fetch the label-to-code mapping
    mapping_dict = label_mappings[target_column]

    # Create reverse map: string label → integer code
    str_to_code = {str(value): key for key, value in mapping_dict.items()}

    # Find the best fuzzy match for the user's input
    match, score = process.extractOne(user_input, str_to_code.keys())

    # Return the encoded integer corresponding to the best match
    return str_to_code[match]
```

```python
# Example use
sample_input = "mid level"
encoded_result = fuzzy_encode(sample_input, 'Job Title')
print(f"Encoded value for '{sample_input}': {encoded_result}")

import re
import nltk
from nltk.corpus import stopwords as sw
from nltk.stem import WordNetLemmatizer as Lemmatizer

nltk.download('stopwords')
nltk.download('wordnet')

# Initialize stopwords set and lemmatizer instance
stop_words = set(sw.words('english'))
lemmatizer = Lemmatizer()
from nltk.corpus import stopwords, wordnet
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
import nltk

# Download required NLTK data (run once)
nltk.download('stopwords')
nltk.download('wordnet')

# Example dataframe - replace this with your actual df
df = pd.read_csv('job_descriptions.csv')

# Preprocessing function for job descriptions
def preprocess_text(text):
    if not isinstance(text, str):
        return ""

    # Lowercase
    text = text.lower()

    # Remove non-alphabetical characters (keep spaces)
    text = re.sub(r'[^a-zA-Z\s]', '', text)

    # Tokenize
    tokens = text.split()

    # Remove stopwords
```

```
    stop_words = set(stopwords.words('english'))
    tokens = [word for word in tokens if word not in stop_words]

    # Lemmatize
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(token) for token in tokens]

    # Join tokens back to string
    return ' '.join(tokens)
# Apply preprocessing to job descriptions
df['Processed_Job_Description'] = df['Job
Description'].apply(preprocess_text)

# Filter out empty descriptions
job_descriptions = df['Processed_Job_Description'].tolist()
job_descriptions = [desc for desc in job_descriptions if desc]

# Vectorize job descriptions
vectorizer = TfidfVectorizer()
job_vectors = vectorizer.fit_transform(job_descriptions)
# Function to match user input to job title based on description
similarity
def match_job_description(user_input):
    user_input_processed = preprocess_text(user_input)
    user_vector = vectorizer.transform([user_input_processed])
    similarities = cosine_similarity(user_vector, job_vectors)
    most_similar_idx = similarities.argmax()
    return df.iloc[most_similar_idx]['Job Title']
```

```
# Function to match user input to job title based on description
similarity
def match_job_description(user_input):
    user_input_processed = preprocess_text(user_input)
    user_vector = vectorizer.transform([user_input_processed])
    similarities = cosine_similarity(user_vector, job_vectors)
    most_similar_idx = similarities.argmax()
    return df.iloc[most_similar_idx]['Job Title']
# Example usage
user_input = "Looking for a mid-level Python developer with experience in
software engineering"
matched_job = match_job_description(user_input)
print(f"Best matched job: {matched_job}")
```